REMAP: A Fine-Grained Runtime Memory Protection System for RTOS

Heeseung Son, BeomSeok Kim, Ben Lee, and Jinsung Cho, *Member, IEEE*

***Abstract*—Embedded systems rely on real-time operating systems (RTOS) for safety-critical applications, but the absence of fine-grained memory protection poses serious security risks. Existing protection mechanisms either require specialized hardware or rely on static configurations that fail to adapt to dynamic task behavior. This article introduces runtime enforcement of memory access protection (REMAP), which is a novel framework that combines LLVM-based static analysis with runtime memory protection unit (MPU) reconfiguration to achieve fine-grained, dynamic compartmentalization for RTOS environments. REMAP constructs task-specific access control table (ACT) at compile time and enforces them at runtime through on-demand MPU updates triggered by memory access violations. This lazy mapping strategy allows REMAP to support hundreds of compartments per task despite the limitation on the number of hardware MPU slots. Our prototype on ARMv8-M (Cortex-M33) demonstrates that REMAP enforces memory access control down to the function and variable level, protects shared buffers for secure inter process communication (IPC), and prevents error propagation across tasks. Evaluation using the BEEBS benchmark suite shows that compartment merging significantly reduces fault frequency, yielding overheads under 5% across all benchmarks. Therefore, REMAP achieves precise, adaptive memory protection suitable for commercial RTOS deployments without compromising real-time performance.**

***Index Terms*—Compartmentalization, embedded security, memory protection unit (MPU), real-time systems, real-time operating systems (RTOS).**

I. Introduction

In RECENT years, embedded systems have shifted from merely serving simple control units to becoming foundational technologies in a variety of domains, including automotive systems, medical devices, industrial automation, and the Internet of Things (IoT) [1]. As these systems become more interconnected and exposed to networked environments, security threats have escalated dramatically requiring immediate attention [2].

With this heightened connectivity, attackers are able exploit vectors, such as malicious firmware updates, remote code execution, and hardware vulnerabilities to compromise devices. Failing to address these evolving threats can lead to severe consequences, such as remote hacking of vehicles, unauthorized manipulation of medical equipment, and large-scale disruptions in industrial systems. Notably, many embedded systems rely heavily on real-time operating systems (RTOS) to handle strict timing constraints and limited resources, which can amplify the impact of security breaches [3]. Consequently, there is an urgent need for robust, RTOS-centric security mechanisms capable of addressing these unique challenges and effectively mitigating vulnerabilities arising from real-time operational requirements [4].

Unlike general-purpose operating systems (e.g., Linux or Windows), which typically employ virtual memory management through hardware features like a Memory management unit (MMU), most RTOS lack such comprehensive memory protection [5]. In full-featured operating systems, each user process operates within its own isolated virtual address space, preventing unwanted access to other processes or kernel-level memory. By contrast, the lightweight design of RTOS, which prioritizes minimal overhead and deterministic performance, often results in shared address spaces across the kernel and application tasks. Although some embedded platforms include a memory protection unit (MPU), its limited number of regions and simpler access-control mechanisms frequently render fine-grained protection impractical. An MMU enforces access permissions and provides virtual-to-physical address translation allowing each process to reside in its own virtual address space, but an MPU generally does not support such comprehensive translation [6]. Instead, protection is managed by defining only a limited set of regions in physical address space. This simplified design reduces overhead, which is a critical factor for real-time systems, but also limits the granularity and flexibility of memory protection. Consequently, tasks in many RTOS deployments can freely access the memory regions of other tasks as well as kernel data structures [7]. This increases the risk of both unintentional data corruption and malicious exploitation, including information leakage and system takeover [8].

In RTOS environments, two critical security issues will arise if strict boundaries between tasks are not enforced. First, multiple tasks often share memory buffers for data exchange. Without robust memory protection, a compromised

Received 5 June 2025; revised 30 July 2025; accepted 16 August 2025. Date of publication 25 August 2025; date of current version 24 October 2025. This work was supported in part by the Ministry of Science and ICT (MSIT), Korea, through the Convergence Security Core Talent Training Business Support Program supervised by the Institute for Information & Communications Technology Planning & Evaluation (IITP) under Grant IITP-2025-RS-2023-00266615. *(Corresponding author: Jinsung Cho.)*

Heeseung Son and Jinsung Cho are with the School of Computing, College of Software, Kyung Hee University, Yongin-si 446-701, Republic of Korea (e-mail: toddlf95@khu.ac.kr; chojs@khu.ac.kr).

Beomseok Kim is with the Department of Artificial Intelligence and Software Engineering, Major in Information Security, Chosun University, Gwangju 61452, Republic of Korea (e-mail: beomseokkim@chosun.ac.kr).

Ben Lee is with the School of Electrical Engineering and Computer Science, Oregon State University, Corvallis, OR 97331 USA (e-mail: benl@eecs.orst.edu).

Digital Object Identifier 10.1109/JIOT.2025.3602576

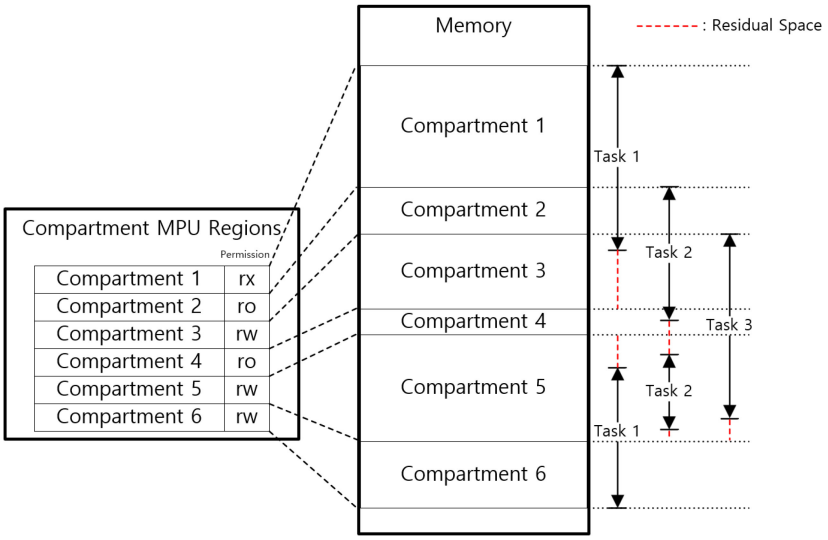


Fig. 1. Practical limitations of memory compartmentalization in embedded systems.

or malicious task can easily intercept cryptographic keys, user credentials, sensor readings, and other sensitive information by simply accessing these shared regions [9]. Second, the lack of isolation among tasks and the RTOS itself allows attackers to modify or overwrite data in other tasks or even in the OS. As such, a single compromised task can take control of the entire system by exploiting vulnerabilities, such as buffer overflows or return-oriented programming (ROP), and trigger critical failures in safety-sensitive domains such as automotive, medical, or industrial control systems [7].

To mitigate these threats, researchers employ a memory protection method known as *compartmentalization* in which memory is partitioned as *compartments* according to a developer-specified policy [10]. Compartmentalization enforces strict runtime access to limit unauthorized interactions and isolate errors, and thus protecting sensitive information. It also preserves deterministic real-time performance by preventing localized faults from escalating into system-wide failures, which is critical in RTOS environments [11]. Moreover, enforcing granular permissions within each compartment allows invalid access attempts to be quickly detected and blocked, mitigating threats, such as ROP and code injection before they can compromise other tasks or disrupt higher-priority operations.

Despite these advantages, effective compartmentalization faces practical limitations, particularly due to the mismatch that often arises between the actual memory usage of tasks and the boundaries of predefined compartments [12]. This is shown in the Fig. 1, where tasks do not align precisely with compartment boundaries resulting in residual spaces—memory regions a task does not use but can still access. These unintended residual spaces compromise isolation by potentially exposing sensitive data to unauthorized access, diminishing the intended security benefits.

Achieving complete compartmentalization in embedded environments is further complicated by limited MPU slots available on many ARM architectures, making fine-grained segmentation challenging. Developers often define only a small number of memory regions, each covering relatively large address spaces. Although this configuration can isolate critical components, such as kernel memory and task stacks, it may not achieve comprehensive separation of code and data at a per-task granularity. Consequently, insufficient MPU regions necessitate tradeoffs, such as consolidating multiple components within a single region or granting broader access permissions than ideally required.

To address these limitations, various approaches have been proposed to enhance security in RTOS-based embedded systems. However, these solutions either rely on static memory protection mechanisms or offer only limited dynamic access control, thus they do not accommodate the dynamic environment of real-time workloads. For example, hardware-based trusted execution environments (TEEs), such as TrustLite [13] and Tytan [14] require modified or new hardware architectures, including redesigned MPUs, to provide protection during execution. Despite this, they still lack the support required for fine-grained access control. Meanwhile, solutions like ACES [10] and MINION [15] rely heavily on static MPU configurations. As a result, they inevitably resort to coarse-grained memory protection strategies due to the limited number of available MPU slots. CompartOS [16] provides dynamic memory protection but requires specialized hardware architectures (e.g., CHERI [17], limiting its applicability in commercial embedded systems. In addition, EPOXY [18] enhances security through static analysis but lacks robust mechanisms for dynamic memory protection. These constraints underscore the necessity for more flexible and fine-grained compartmentalization techniques that can effectively support dynamic tasks and diverse requirements in RTOS environments.

To overcome the aforementioned limitations, this article proposes a fine-grained method called *runtime enforcement of memory access protection (REMAP)* for dynamically controlling memory access within RTOS environments. REMAP integrates LLVM-based static analysis with runtime MPU management to regulate task-specific memory privileges in

real time. The proposed REMAP first generates a control flow graph (CFG) for each RTOS task using LLVM to identify the memory regions and corresponding access permissions required. This information is compiled into an access control table (ACT), which defines permissible memory segments and access permissions for each task.

REMAP leverages dynamic MPU updates at runtime to enforce these access controls. Whenever a task attempts to read or write a memory region not currently registered in the MPU, a Memory Management Fault exception (MemManage exception) is triggered [19]. The exception handler then checks whether the accessed region is present in the ACT of the executing task. If the region is defined as accessible, the MPU configuration is updated to reflect the allowed memory range and permissions to enable uninterrupted task execution. Otherwise, unauthorized accesses are blocked.

In contrast to static or coarse-grained solutions, such as ones presented in [10], [13], [14], [15], [16], [18], REMAP provides a dynamic and fine-grained level of protection by compartmentalizing down to the function and global variable level. This approach allows more precise control over memory accesses within RTOS environments to provide flexible runtime memory protection, thus enhancing both security and system reliability.

The proposed REMAP framework offers several key contributions that advance the security and reliability of RTOS-based embedded systems.

1) *Fine-Grained Memory Protection in RTOS:* While prior RTOS implementations often provided limited task-level protection, REMAP implements a dynamic allocation of MPU slots to achieve more precise memory compartmentalization. By dynamically configuring the MPU, individual tasks are effectively isolated to prevent unauthorized access to or manipulation of neighboring memory regions.
2) *Implements a Strict Separation of User and Kernel Modes:* Many existing RTOS environments do not rigorously differentiate between user mode and kernel mode. REMAP introduces a clear separation of privilege levels to prevent tasks from freely accessing kernel memory regions. The result is a more robust boundary enforcement that mitigates the risk of system-wide compromises originating from a single vulnerable task.
3) *Secure IPC Mechanism:* The proposed REMAP enforces strict MPU protections on shared buffers to ensure that only authorized tasks can access inter process communication (IPC) memory segments. This allows secure and dependable data exchange in embedded systems where shared memory serves as the primary mechanism for IPC.
4) *Preventing Error Propagation Between Tasks:* Finally, the systematic use of MPU controls at the task level helps localize errors—whether malicious or accidental—to the originating task. Consequently, even if a task encounters an unrecoverable fault, it does not destabilize or crash the entire system, thus maintaining overall reliability in real-time settings.

The remainder of this article is organized as follows. Section II provides background on the ARM MPU, its integration with FreeRTOS, and reviews related work on memory protection in embedded systems. Section III presents the design of REMAP, detailing its LLVM-based static analysis, dynamic memory enforcement mechanism, and optimization strategies, such as compartment merging. Section IV describes the implementation and experimental setup, and evaluates REMAP in terms of memory usage, runtime overhead, benchmark performance, and real-time applicability. It also compares REMAP with prior approaches and discusses practical limitations. Finally, Section V concludes this article and discusses possible future work.

II. Background

A. MPU

An MPU is a hardware feature in ARM-based microcontrollers that enforces access permissions on defined memory regions, preventing unauthorized or unintended operations by restricting which parts of memory each task can access [20]. In an embedded environment—particularly when running a RTOS—this protection mechanism significantly enhances reliability and security by containing faults and mitigating malicious behavior at runtime.

While earlier ARM architectures, such as ARMv7-M required memory regions to be sized in powers of two, ARMv8-M microcontrollers (e.g., those based on the Cortex-M33 or Cortex-M23) can define regions in multiples of 32 bytes. This improvement provides far greater flexibility in carving out memory boundaries, allowing system designers to align protection regions more closely with actual task requirements. The MPU in ARMv8-M typically supports up to eight configurable regions (though specific implementations may vary), each of which can be assigned attributes, such as read/write permissions and execution permissions (e.g., the XN bit) [19].

This article focuses specifically on the ARMv8-M MPU and leverages its fine-grained memory partitioning capabilities. By combining the flexible region definition of the MPU with static analysis performed via LLVM, it becomes possible to generate an ACT that precisely specifies which regions each task can access, and with what permissions. During runtime, the RTOS updates MPU configurations dynamically whenever a task attempts to access memory outside its currently active region, ensuring a level of protection that approaches that of full-function operating systems.

B. FreeRTOS-MPU

FreeRTOS-MPU is an extension of the FreeRTOS kernel that leverages the hardware MPU in many ARM Cortex-M microcontrollers to provide a basic form of memory protection. Under this scheme, each task can be assigned a set of MPU regions with specific access permissions. Tasks may also be configured to run either in privileged mode, with unrestricted system access, or in unprivileged mode, with access restricted to the assigned regions. By design, this helps mitigate the risk of tasks inadvertently or maliciously accessing memory

outside their designated areas, thereby improving system robustness and security compared to a purely software-based RTOS approach [21].

Despite these benefits, FreeRTOS-MPU has several inherent limitations [22]. First, tasks are often allocated only a small number of MPU regions, which makes it difficult to assign isolation at the level of individual functions or data objects. This coarse-grained approach can be sufficient for simple applications, but it may become restrictive in more sophisticated or safety-critical systems that demand fine-grained isolation. Second, the configuration of MPU regions in FreeRTOS-MPU is typically performed statically at compile-time, limiting the ability of the system to adapt to changing memory requirements during runtime. Dynamic updates to MPU configurations, if needed, must be carefully implemented and can introduce additional complexity. As a result, while FreeRTOS-MPU represents a significant step forward for embedded systems running FreeRTOS, it still lags behind full-featured operating systems, such as Windows or Linux in terms of the depth and flexibility of memory protection it can offer.

C. Related Work

Embedded systems, particularly those utilizing RTOS, have inherent constraints in hardware resources and strict timing requirements. These limitations often lead to minimal or lack of memory protection mechanisms compared to general-purpose operating systems. To address such security gaps, numerous studies have explored compartmentalization and memory-protection methods for RTOS-based embedded environments, which are categorized into hardware-based approaches, static-analysis-based methods, and dynamic memory-protection schemes.

Among hardware-based methods, Koeberl et al. introduced TrustLite [13], which is a security architecture that combines microkernel structures with hardware extensions to provide TEEs for small embedded devices. TrustLite establishes isolated contexts by employing hardware-enforced memory boundaries to significantly improve security over purely software-based solutions. However, its protection policies are statically configured at compile-time, limiting runtime flexibility and adaptability to dynamically changing execution requirements. Extending this concept, Brasser et al. proposed Tytan [14], which added support for event-based security policies tailored specifically for RTOS environments. Despite its improved support for real-time execution contexts, Tytan retains the static MPU configurations and thus lacks the ability to dynamically manage memory protections during runtime. Additionally, recent advancements like CompartOS by Almatary et al. [16] leverage the CHERI [17] architecture to achieve dynamic memory protection in RTOS environments. Although CompartOS provides more sophisticated dynamic protections, its dependency on the specialized CHERI hardware significantly limits practical deployment in standard commercial embedded systems. Moreover, CompartOS primarily provides task-level memory compartmentalization, lacking a finer granularity support, such as function-level or global-variable-level memory protections.

Static-analysis-based approaches represent another prevalent strategy for securing RTOS environments. Clements et al. [18] developed EPOXY, which is a system leveraging static analysis to predefine memory access patterns of each task at compile-time to enforce static integrity checks and memory isolation. However, EPOXY does not adapt to runtime changes, making it inadequate for dynamic memory access scenarios. Following this approach, the same authors introduced ACES [10], which static analysis to automate MPU configurations to improve task isolation. Although ACES enhances automation in RTOS security, it is constrained by hardware limitations, particularly the small number of MPU slots available on typical microcontrollers, resulting in coarse-grained memory protection. Furthermore, it is unable to protect dynamically allocated memory regions, such as heap areas, restricting its use in complex real-world applications. Another recent study called CRT-C [23] utilizes LLVM and the CheckedC [24] type system to compartmentalize RTOS firmware at variable granularity, significantly improving type-based memory safety. Nevertheless, the protection mechanisms of CRT-C remained strictly compile-time-oriented without runtime adaptability, rendering it ineffective in dynamic environments. Similarly, Khan et al. [25] proposed EC, which introduced a compartmentalization framework that combines LLVM-based static code partitioning with MPU and data watchpoint and trace (DWT)-based runtime memory access enforcement. Despite enhancing compatibility with existing RTOS codebases and providing intrakernel isolation, the protection granularity of EC remains coarse (thread- or file-level). This prevents memory accesses to be precisely manage at finer granularities, such as functions or global variables.

Dynamic MPU-management approaches represent a further evolution in compartmentalization strategies. MINION by Kim et al. [15] was an early attempt in this direction, which dynamically configures MPU regions at runtime to reinforce task-level memory isolation. While MINION provided more flexible protections than purely static approaches, it retains a relatively coarse granularity at the task level making precise memory access control challenging. In parallel, OPEC [26] instruments security-sensitive operations to trigger MPU reconfiguration on demand, thereby tightening access windows around those operations. However, similar to MINION, OPEC does not systematically manage dynamically allocated heap objects or fine-grained intratask regions, and thus its protection remains limited in highly dynamic execution contexts.

In summary, prior research in RTOS memory protection shares several notable limitations. First, static methods lack runtime adaptability and cannot accommodate dynamic memory-access patterns in real deployments. Second, hardware-based solutions often require specialized components or architectures not readily available in commercial embedded devices, severely limiting their practical adoption. Lastly, many approaches are limited by coarse granularity due to inherent MPU-slot constraints, which hinders their ability to provide fine-grained memory protections down to individual functions or data objects. Therefore, there remains a large gap in achieving fine-grained, flexible, and dynamically adaptable

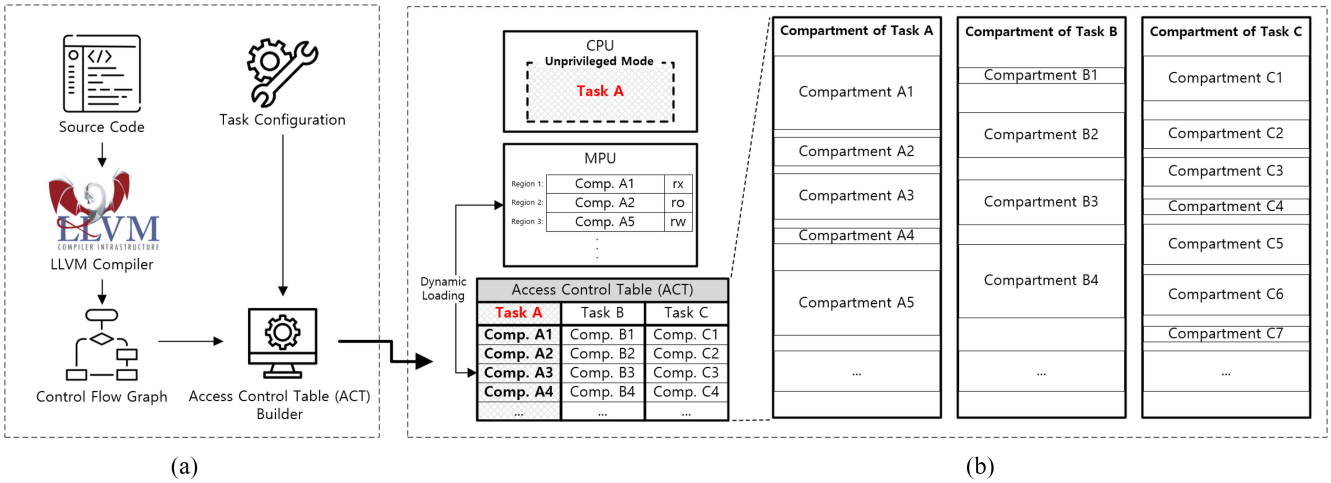


Fig. 2. REMAP overview. (a) LLVM analysis. (b) Runtime.

memory protection mechanisms suitable for deployment in standard commercial RTOS environments.

III. Threat Model and Security Guarantees

This section presents the threat model, security objectives, and guarantees of the proposed REMAP system. It also identifies residual attack surfaces, such as hardware-based threats, which are beyond the scope of our defenses but remain important for understanding the overall threat model of the system.

REMAP assume a software-only adversary capable of hijacking control flow within unprivileged tasks. The attacker may exploit memory corruption vulnerabilities, such as buffer overflows, use-after-free, or format string bugs to achieve arbitrary code execution. Once a task is compromised, the adversary may attempt to access any physical memory address, including the code or data of other tasks, RTOS kernel memory, and shared IPC buffers. Additionally, the attacker is assumed to possess the ability to construct advanced payloads, including ROP or jump-oriented programming (JOP).

REMAP is designed to mitigate memory-based threats arising from such compromised tasks. Its primary security objectives are as follows: 1) *memory isolation*, which prevents compromised tasks from accessing memory regions outside their authorized boundaries; 2) *privilege separation*, which enforces strict division between user-mode tasks and privileged kernel memory; 3) *secure IPC* which ensures that only authorized tasks can access shared memory segments; and 4) *fault isolation*, which restricts the propagation of faults to within the faulty task, preserving overall system stability.

However, REMAP does not address physical or hardware-level attacks. Scenarios involving invasive hardware probing, voltage or clock glitching, fault injection, or microarchitectural side channels (e.g., timing or power analysis) are beyond the scope of this article. The MPU is assumed to operate correctly and cannot be bypassed through undocumented behavior or hardware flaws. Similarly, peripheral devices with direct memory access (DMA) capabilities are considered trustworthy and constrained to preconfigured memory regions.

This adversary model reflects realistic attack vectors frequently encountered in embedded systems, particularly those connected to networks or exposed to remote inputs. REMAP is designed to enforce isolation and integrity in such contexts, where software-only attacks pose significant threats to system reliability and security.

IV. Proposed REMAP System

A. Overview

This section proposes REMAP, which a fine-grained, dynamically applicable memory protection mechanism specifically for RTOS environments. Fig. 2 shows the REMAP mechanism. Fig. 2(a) shows the static analysis phase, where LLVM is utilized to identify and track the functions and global variables required by each task. Based on this analysis, an ACT is constructed. The ACT lists the memory regions each task is authorized to access and their corresponding permissions (read, write, execute). Although much of the ACT can be determined at compile-time, additional entries are populated at runtime to reflect dynamically allocated memory regions and evolving access requirements.

Fig. 2(b) shows how the ACT controls memory access at runtime. Due to the limited number of MPU slots, REMAP adopts an on-demand (lazy mapping) approach. When a task begins execution, only a minimal set of essential memory regions is initially loaded into the MPU. If the task attempts to access additional memory defined in its ACT but currently unmapped, the hardware triggers a MemManage exception. The exception handler then checks whether the requested access corresponds to a legitimate memory region in the ACT. If the requested address is valid, the MPU configuration is dynamically updated to permit that access. As a result, the MPU operates similarly to a cache that selectively protects only those memory regions actually used by each task. Moreover, during task switches, the RTOS saves the MPU configuration for each task, which is then restored upon rescheduling. This mechanism enables each task to retain access to its previously mapped memory regions, thereby avoiding redundant faults for these regions. It also ensures that

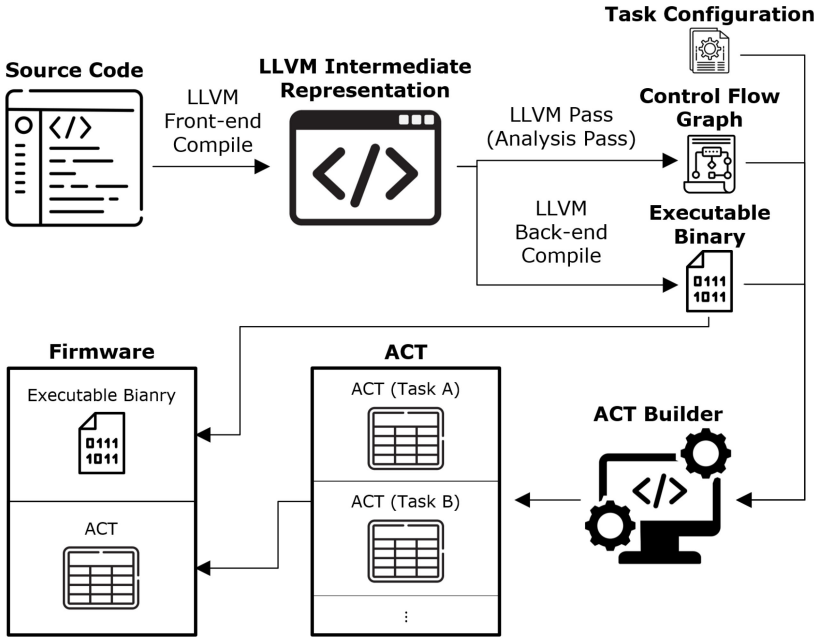


Fig. 3. REMAP development process.

the next scheduled task inherits no residual permissions. As a result, REMAP consistently preserves the memory protection state of each task and maintains strong isolation across task boundaries.

The remainder of this section is organized as follows: First, Section IV-B describes the LLVM-based construction of the ACT. Then, Section IV-C outlines the runtime exception-handling workflow with the corresponding MPU reconfiguration strategy. Finally, Section IV-C1 presents how dynamically allocated memory regions are integrated into a unified ACT structure, and summarizes the principal implementation considerations.

B. LLVM-Based Analysis

REMAP constructs the ACT via an LLVM-based static analysis performed at compile-time. The development process is shown in Fig. 3. During this process, each RTOS task is analyzed to determine all the memory regions it may legitimately access. In most cases, an RTOS task is defined by a task entry function; thus, the analysis starts at this entry point and traverses all reachable code and data. Using the LLVM pass framework, the firmware is instrumented to generate a CFG for each task. This CFG captures every function as well as all global and static variables that these functions may access.

By traversing the CFG of a specific task, the static analysis identifies all global variables, functions, and memory-mapped resources that the task may read, write, or execute. This procedure provides a comprehensive and fine-grained view of each task's associated memory regions at the level of individual symbols or objects. For example, suppose Task *A* invokes a library function that accesses a global variable *G*. In that case, the analysis infers that Task *A* requires read or write privileges to the memory region associated with *G*. Similarly, any code segment that Task *A* might execute, such as its own functions and any library or driver functions it calls, is flagged as executable for that task. The result of this stage is a preliminary access list per task, specifying the memory segments and the corresponding access permissions (e.g., read-only, read-write, or execute).

Following the static analysis, the ACT builder tool processes the gathered information to generate the ACT entries. This tool aggregates three primary sources of input: 1) the per-task memory object list and access permissions derived from the CFG analysis (which are obtained through an LLVM analysis pass); 2) the symbol table from the compiled executable binary, which provides actual address and size information of functions and global variables; and 3) any metadata (e.g., developer annotations) provided in task configuration files for special-purpose memory regions that cannot be discovered by static analysis alone (e.g., memory-mapped device registers). Using these inputs, the tool maps each identified memory object to its corresponding base address and size in the final program binary and assigns the appropriate access permissions. The resulting output is a complete ACT entry for each task, listing the start addresses, lengths, and permitted access types of all authorized memory regions.

Once generated, the ACT is embedded into the system firmware as a constant data structure stored in flash memory to be referenced at runtime. By performing the analysis at compile-time, REMAP achieves highly fine-grained memory partitioning for each task, down to the level of individual functions or global variables. This level of granularity would be difficult to achieve through manual MPU configuration. Moreover, because the ACT is created before running, it incurs no additional overhead to the running system. The only runtime overhead arises from enforcing these access permissions, which involves handling memory faults and reconfiguring the MPU when necessary, as discussed in the following section.

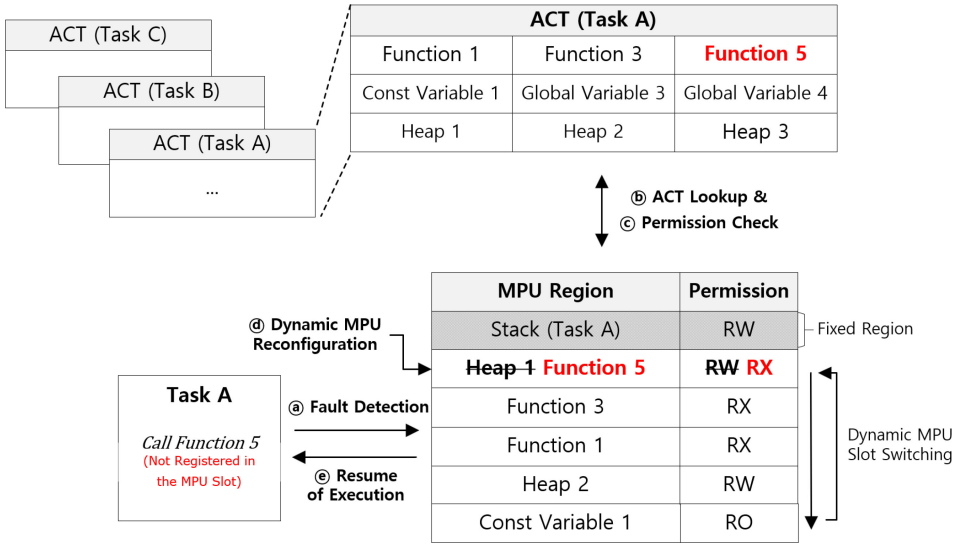


Fig. 4. REMAP runtime operation.

C. Runtime Memory Protection

REMAP employs the ARM MPU at runtime as an access control mechanism to enforce the memory access policies defined in the ACT. A key principle of the REMAP runtime architecture is that permitted memory regions are loaded into the MPU on demand, rather than map them statically. When a task begins execution, the RTOS initializes the MPU with only a minimal set of essential regions. Typically, this includes the code region corresponding to the current program counter of the task and the stack region. All other memory regions authorized by the ACT remain unmapped initially. This on-demand mapping strategy ensures that the MPU contains only a subset of the permitted regions for the task, specifically those currently in use. As such, REMAP effectively manages the limited set of region slots in the MPU to accommodate a larger number of protected memory regions.

If a task attempts to access a memory region not currently mapped in the MPU, the hardware triggers a fault causing the dynamic mechanism of REMAP to handle the access. Through this process, the system gradually extends the accessible memory view of the task, mapping only those regions essential for correct execution. The following sections detail how REMAP manages memory access faults at runtime and how the memory protection context for each task is preserved during multitasking context switches.

1) Memory Access Fault and Handling: When a task attempts to access an address not configured in the MPU, a *MemManage* exception (Memory Management Fault) occurs. This exception signifies a permission violation defined through the MPU. To facilitate on-demand region loading, REMAP registers a custom MemManage handler. Fig. 4 shows the fault handling procedure in five distinct stages: Fault Detection, ACT Lookup, Permission Check, Dynamic MPU Reconfiguration, and Resume of Execution.

1) *Fault Detection:* Upon detection of an invalid memory access, the processor immediately invokes the MemManage exception handler within the RTOS kernel. The exception mechanism provides the handler with diagnostic information, including the faulting address and the cause of the exception. Since the task was executing in unprivileged mode when the fault occurred, the processor transitions into privileged mode upon entering the exception handler, thereby granting the kernel the necessary privileges to reconfigure the MPU and manage the system state.
2) *ACT Lookup:* The handler then identifies the task that caused the fault. This is done by either referencing a pointer to the control block for the current task or extracting an identifier from the exception context, which is then used to refer to the corresponding entry in the ACT associated with the task. The ACT lookup examines the set of authorized memory regions for the task to determine whether any entry covers the faulting address. Specifically, the system verifies if the faulting address is within a valid memory region assigned to the task, and whether that region permits the attempted access type (read, write, or execute). This procedure differentiates between accesses that are legitimate according to the ACT but not currently mapped in the MPU, and those that are completely unauthorized for the task.
3) *Permission Check:* If the ACT lookup does not locate any entry covering the faulting address, the memory access is deemed unauthorized. In such cases, the handler denies the access and treats it as a security violation. Depending on the security policy of the system, the kernel may perform protective measures, such as logging the incident, raising a security alert, or terminating the malicious task. Most importantly, the MPU configuration remains unchanged in this scenario, thus prohibiting the task from accessing that memory region.
4) *Dynamic MPU Reconfiguration:* If the ACT lookup confirms that the faulting address resides within a memory region assigned to the task, REMAP resolves the exception by dynamically reconfiguring the MPU to

include the corresponding region. During this process, REMAP allocates an MPU region slot to the target memory segment. If an unused slot is available, it is assigned immediately. Otherwise, an existing region must be removed to free a slot. REMAP adopts a simple first-in, first-out (FIFO) replacement policy by treating the MPU as a cache of recently accessed regions and selecting the region that has remained unchanged for the most extended period. This strategy minimizes implementation complexity and management overhead while offering predictability and constant-time selection, which make it appropriate for resource-constrained embedded systems. However, REMAP protects certain critical regions from eviction to ensure system stability. These include the region containing the faulting instruction and the stack region allocated to the task, both of which are essential for correct execution. Evicting them would likely result in immediate refaults upon handler return. Consequently, the replacement algorithm excludes these critical entries and evicts the next available region in FIFO order. After obtaining a usable slot, REMAP reprograms the MPU with the corresponding base address, size, and access permissions (e.g., read/write or execute) for the new region, which are derived from the ACT.

5) *Resume of Execution:* After reconfiguring the MPU, the exception handler completes and the RTOS restores execution of the faulting task at the instruction where it originally halted. Since the corresponding memory region has been mapped with the required access permissions, the task continues operating as if the access were authorized from the outset. Except for the minor delay introduced by the exception handling process, the task experiences no observable disruption. The MPU update occurs seamlessly, i.e., the program does not need to request access to memory regions explicitly, and its functional behavior remains unchanged.

The aforementioned on-demand memory mapping mechanism allows the memory view of each task to dynamically adapt to actual access patterns, yet remains bounded by the ACT. This allows fine-grained memory isolation beyond the intrinsic limit of the fixed region slots in the hardware. The primary overhead introduced by this mechanism arises from handling MemManage exceptions and executing MPU reconfiguration. However, as shown in the performance evaluation, this cost remains modest and is outweighed by the substantial benefit. REMAP supports dozens or even hundreds of protected memory compartments per task, even though the MPU hardware permits only a limited number of simultaneous regions.

2) MPU Restoration on Context Switching: In a multitasking RTOS, the scheduler periodically performs context switches to distribute CPU time among multiple tasks. REMAP extends the standard context switching mechanism by performing the saving and restoring of MPU configurations on a per-task basis. As a result, every task resumes execution under the correct MPU configuration, maintaining consistent and secure enforcement of the memory protection policy.

In conventional systems that use static MPU configurations on a per-task basis, such as FreeRTOS with MPU support, the operating system typically loads a predefined set of memory regions for a task when it is scheduled. These regions are often specified during compilation and stored in a fixed configuration table. In contrast, REMAP maintains a flexible set of MPU regions for each task that adapt at runtime, i.e., when a memory access fault occurs, new regions are loaded and others evicted.

To accommodate this adaptive behavior, REMAP incorporates the MPU state as part of the runtime context of the task. Therefore, the RTOS captures a snapshot of its current MPU configuration when a task is preempted, specifically the set of memory regions presently mapped for that task along with their corresponding permissions. This snapshot is stored in the task control block (TCB) or an equivalent data structure tied to the task. The snapshot may be represented by either saving the values of MPU registers directly or maintaining an updated list of active region descriptors.

During a context switch, the MPU configuration of the outgoing task is first saved into its TCB, and the saved MPU configuration of the incoming task is then restored. Specifically, when outgoing Task *A* is replaced by incoming Task *B*, the kernel stores the current list of active MPU regions for Task *A*, capturing the set of memory regions Task *A* could access at the moment of suspension, and subsequently reprograms the MPU with the region list previously recorded by Task *B*. As a result, when Task *B* resumes execution, the MPU is already configured to permit access to the memory regions it utilized during its last execution period. The task continues with the same access permissions it held prior to preemption, avoiding additional memory faults and eliminating the need to reload previously granted regions. In this way, the MPU configuration is treated as an integral part of the runtime context of the task, analogous to general-purpose registers or the stack pointer, and is seamlessly restored during the context switch process.

Restoring the MPU state during context switches provides two primary benefits. First, it avoids redundant fault handling for memory regions that a task frequently accesses. If a task has already incurred handling a MemManage exception to load a particular memory region in a prior execution period, requiring the task to fault again immediately upon resuming would be inefficient. By restoring the MPU configuration for the task, REMAP ensures that previously accessed regions are available when the task resumes execution. For instance, consider a scenario where Task *A* frequently accesses a particular global array. The first time Task *A* references the array, a memory fault triggers REMAP to load the corresponding memory region into the MPU. If Task *A* is subsequently preempted and later scheduled again, REMAP restores the memory region in the MPU prior to execution of Task *A*. As a result, Task *A* can promptly access the global array without triggering another fault, as its access permissions have been preserved.

Second, this mechanism enhances memory isolation by eliminating any possibility that a task inherits residual MPU settings from another task. When Task *B* is scheduled after

Task *A*, REMAP ensures that only the memory regions authorized for Task *B* are loaded into the MPU. Any MPU entries associated with Task *A* are cleared during the context switch. As a result, Task *B* cannot accidentally or maliciously access memory regions exclusively intended for Task *A*. As such, each task operates with a dedicated set of memory access permissions specific to its execution context.

From a performance perspective, incorporating the MPU state into the context-switching process is both efficient and critical for maintaining real-time responsiveness. Restoring the MPU with a previously saved configuration, typically involving a small number of register writes per region (e.g., 8 to 16 regions in common use cases), is a fast and deterministic operation. This restoration takes place during context switches, which are well-defined control points in execution, and the number of MPU entries fully bounds its cost. This mechanism is notably more efficient than rehandling a series of memory faults that would otherwise be needed for each task to rediscover and reload its required memory regions every time it resumes. By eliminating the necessity to start from an empty memory permission set at each scheduling event, REMAP substantially reduces the worst-case overhead associated with memory protection in a time-multiplexed system.

This design effectively combines fine-grained memory isolation with rigorous timing constraints of RTOS. It shares conceptual similarities with prior work where the memory view of a task is treated as part of its execution context in systems that save and restore protection domains during context switches [15]. However, REMAP extends prior approaches by supporting an adaptively evolving memory view per task.

3) ACT for Dynamic Memory Allocation: A notable limitation of statically configured memory protection mechanisms (as discussed in Section IV-B) is their inability to accommodate memory regions that are allocated dynamically during program execution. In many RTOS-based embedded systems, tasks frequently obtain additional memory through heap allocation interfaces (e.g., *malloc*), where the exact base address and size of the memory block are known only at runtime. As a result, it is impractical to enumerate the complete memory footprint of each task at compile-time. To address this limitation, REMAP extends the ACT at runtime by dynamically registering newly allocated memory regions and their corresponding access permissions. This unified ACT structure allows both statically known and dynamically acquired memory segments to be managed under a single access control framework, ensuring consistent and fine-grained memory isolation throughout execution.

Upon calling a dynamic memory allocation routine, REMAP intercepts the allocation event and identifies the allocated address range and its size. The allocated region is then registered into the ACT with the appropriate access permissions (e.g., read/write). This ensures that dynamically allocated memory segments are incorporated into the same compartmentalization framework as statically defined regions, maintaining consistent memory protection semantics throughout execution.

When a memory block is deallocated (e.g., using *free*), REMAP removes the corresponding ACT entry and invalidates any active MPU mappings associated with the region. Any subsequent access attempts to the deallocated memory trigger a MemManage fault, thereby preventing unauthorized reuse or abuse of freed memory.

By unifying the handling of both static and dynamic memory regions under a single ACT structure, REMAP ensures robust and fine-grained memory isolation across the entire address space. This approach is particularly effective in RTOS environments where memory requirements evolve at runtime, enabling secure and adaptive enforcement of task-specific access control policies.

To properly support heap objects, REMAP provides wrapper APIs (e.g., *REMAP_malloc()* and *REMAP_free()*) that transparently interpose on dynamic allocation routines. When a task requests memory through *REMAP_malloc()*, the allocator records the returned address range and its size, and immediately registers this region into the ACT of the task with appropriate permissions. Conversely, *REMAP_free()* removes the corresponding entry from the ACT and invalidates any active MPU mappings for that region so that subsequent accesses to the freed block raise a MemManage fault. This API-level indirection enables REMAP to maintain a unified ACT that seamlessly tracks both statically known objects and dynamically created ones, ensuring that fine-grained isolation is preserved even as memory demands evolve at runtime.

D. Compartment Merging Optimization

While fine-grained memory isolation is desirable, mapping every function or variable to its own MPU region is infeasible on an ARMv8-M target because only a handful of region slots are available. As shown in Fig. 5(a), the straightforward mapping for Task *A* occupies four slots, and a task that later accesses dozens of such objects inevitably incurs frequent region evictions and MemManage exceptions.

REMAP mitigates this limitation through the compartment-merging technique shown by the transition from Fig. 5(a) to (b). This is achieved by combining contiguous memory ranges that share identical permissions into a single ACT entry, thereby reducing the number of regions instantiated at run time. Merging is allowed only when 1) the regions are adjacent (or separated solely by alignment padding) and 2) their access attributes are identical (e.g., RX or RW). Regions spanning different tasks or demanding heterogeneous permissions are never merged, thus preserving isolation semantics.

A practical implementation can leverage linker-script directives to improve region co-location, such as grouping code and data belonging to a task into adjacent memory areas. Such organization leads to more effective merging, allowing the four ACT entries in Fig. 5(a) to be consolidated into the two clustered entries shown in Fig. 5(b). The clustered ACT lowers the frequency of MPU reconfigurations and reduces fault-handling overhead. Empirical measurements confirm that, under the conservative merging policy, runtime faults drop markedly without eroding security guarantees.

E. Additional Considerations for REMAP

In designing REMAP, several practical considerations emerged due to the specific characteristics of RTOS environments and the architectural constraints of ARM MPUs. This

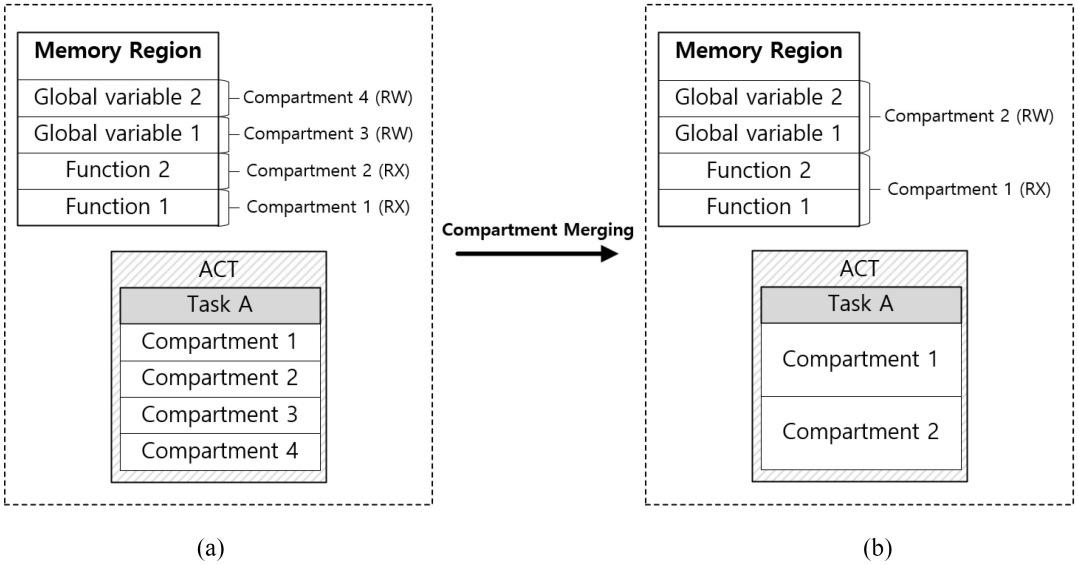


Fig. 5. Compartment merging. (a) Before merging. (b) After merging.

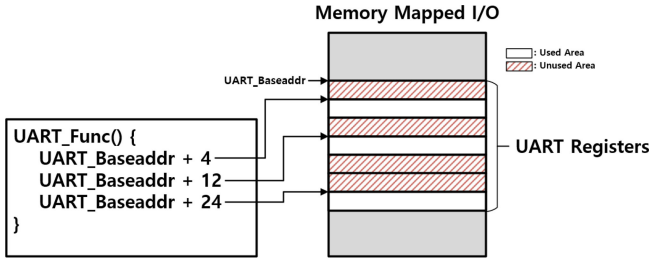


Fig. 6. Memory mapped I/O access example.

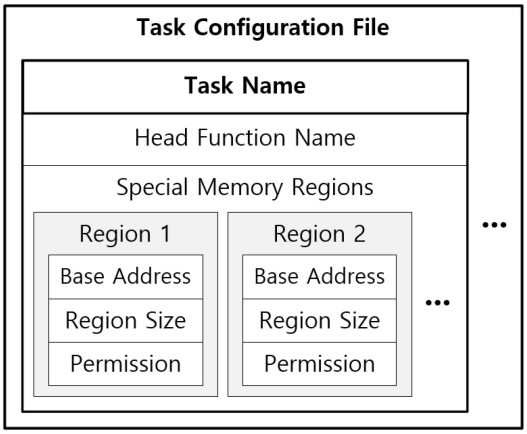


Fig. 7. Task configuration file.

section describes how these considerations were addressed, focusing on aspects, such as task configuration and region alignment constraints.

1) Task Configuration: One significant challenge in REMAP is accurately identifying the set of tasks and their memory usage through static analysis. Unlike traditional process-based systems, tasks in an RTOS are often instantiated programmatically (e.g., through API calls), and may share functions or data structures in complex ways. This introduces ambiguity when attempting to infer task boundaries and resource usage automatically. For instance, in systems like FreeRTOS, tasks are frequently created by passing a function pointer to the scheduler. While static analysis can identify the code of the function, it may lack sufficient context to determine that the function constitutes a distinct task with its own stack and access privileges.

Similarly, memory-mapped peripherals typically occupy a contiguous address range, with specific subregions allocated for different control or status registers. Task code often interacts with these registers through direct pointer arithmetic without referencing the entire register layout. This results in an implicit and fragmented access pattern that complicates static analysis. For instance, as shown in Fig. 6, *UART_Func()* accesses only three specific registers at offsets 4, 12, and 24 from the base address, leaving the majority of the memory-mapped region unused. From a static analysis perspective, this makes it difficult to accurately infer which parts of the peripheral are actually required by the task. Consequently, indiscriminate inclusion of the entire peripheral range would unnecessarily inflate the task access permissions, whereas over-restriction might result in runtime faults.

To address these limitations, REMAP incorporates a user-defined JSON configuration file that supplements the static analysis with explicit task information. As shown in 7, each entry records the task identity (“Task Name”), its entry function (“Head Function Name”), and an extensible list of special memory regions (“Special Memory Regions”). Each region item explicitly encodes the base address, region size, and permission fields. In essence, the configuration file acts as a developer-assisted annotation of the intended compartmentalization of the system. For example, if Task *B* is expected to interact with a sensor mapped to a specific address range, that range is listed in the configuration and incorporated into the ACT for Task *B* by the ACT builder. This approach resolves ambiguities that static analysis alone may fail to capture

and decouples REMAP from RTOS-specific task-instantiation mechanisms.

In the implementation, most memory usage could be inferred automatically. Only a limited number of exceptions, primarily those related to hardware-mapped registers, required manual specification. This configuration-driven methodology allows REMAP to be applied across diverse RTOS platforms without requiring extensive modifications to the underlying static analysis framework.

2) Region Alignment Constraints: Another practical challenge in implementing fine-grained memory protection with REMAP arises from the hardware constraints of the ARMv8-M MPU. The ARMv8-M architecture requires MPU region start addresses to be aligned to 32-byte boundaries and region sizes to be multiples of 32 bytes. Consequently, since functions and global variables are often laid out contiguously in memory, isolating them as separate MPU regions under the 32-byte alignment and sizing constraints may inadvertently include adjacent objects, thereby violating intended memory boundaries.

For example, consider a global variable that occupies 20 bytes. Since the base address of the variable is not 32-byte aligned, the corresponding region will include adjacent memory, potentially exposing unrelated variables that the task is not authorized to access.

To mitigate this risk, REMAP ensures that all protected memory compartments are aligned and sized to fit precisely within MPU-compliant boundaries. This is achieved by modifying the linker script and applying compile-time alignment directives. Functions and global variables are padded or reordered such that their starting addresses are 32-byte aligned, and their sizes are rounded up to the nearest multiple of 32 bytes. Any padding added is either left unused or filled with benign data to prevent leakage of sensitive information and maintain isolation guarantees.

This alignment strategy enables ACT entries to correspond exactly to the boundaries of their associated objects, eliminating the risk of unintentional overpermission. While this approach introduces slight memory overhead due to padding, the impact is minimal, typically less than 31 bytes per object and often negligible for larger or naturally aligned data structures. Moreover, the MPU in ARMv8-M, which supports region sizes in 32-byte increments (unlike the power-of-two size restrictions in earlier ARM MPUs), further minimizes memory waste.

V. Evaluation

The REMAP prototype was implemented on an STM32L562E-DK development board featuring an ARM Cortex-M33 CPU (ARMv8-M architecture) running at 110 MHz with 512 KB of on-board flash and 256 KB of SRAM. The firmware runs a customized FreeRTOS v10.6.1 extended with the proposed REMAP scheme. All the codes were compiled using ARM Clang (LLVM 17.0.0) with full optimizations and applied with linker script modifications to enforce 32-byte alignment of protected regions. All performance measurements were obtained using the processor DWT unit to acquire precise cycle counts for timing-critical operations. This hardware-based profiling ensured minimal measurement overhead and high timing accuracy for our evaluation.

TABLE I
Context Switching Overhead

Configuration	Context Switch (clocks)	Task Execution (clocks)	Duty Cycle
MPU OFF	1,832	196,168	99.07%
MPU ON	3,768	234,956	98.42%

A. Performance Evaluation

1) Flash Memory Overhead: The fine-grained compartmentalization introduced by REMAP causes a slight increase in flash usage due to alignment padding and metadata. The base system firmware occupied 3 29 128 bytes of flash, whereas the REMAP-enabled firmware occupied 3 44 640 bytes—an increase of about 4.8%. This overhead stems primarily from the padding required to meet MPU alignment constraints. The ACT structure is compact requiring only 344 bytes for our test workload and thus contributes negligibly to flash usage. In practice, the code and static data padding overhead (typically at most 31 bytes per protected object) remains modest and well under 5% of total flash, which is a cost outweighed by the benefits of finer isolation.

2) RAM Usage Overhead: Similar to flash memory overhead stack, global variables and heap allocations are aligned to 32-byte boundaries, incurring some padding overhead in memory allocation. The additional padding for data sections and dynamic allocations leads to an average of around 5% RAM overhead. Our experiments show that the increase in SRAM usage was minimal. This indicates that fine-grained memory protection under REMAP can be applied without significant memory pressure in typical embedded scenarios.

3) Context Switch Overhead: The runtime cost of REMAP MPU reconfiguration during context switches was evaluated by measuring the fraction of CPU time spent in the RTOS scheduler with and without MPU protection. Table I summarizes the results. With MPU protection disabled, the system required 1832 cycles per context switch, resulting in a task execution duty cycle of approximately 99.07%. When REMAP MPU protection was enabled, the context switch cost increased to 3768 cycles, reducing the CPU duty cycle to 98.42%. In other words, the additional operations required to save and restore MPU regions during each context switch introduced approximately 0.65% overhead in terms of CPU utilization. This impact remains minimal and the scheduler still allocates over 98% of CPU time to application tasks. This indicates that the context switching overhead introduced by REMAP is negligible for real-time scheduling.

4) MemManage Fault Handling: REMAP incurs a runtime cost when a task first accesses a protected memory region not yet loaded into the MPU, which results in a MemManage exception. The MemManage exception handler was instrumented to measure its execution time. Each fault triggers the handler to look up the faulting address in the task ACT and update the MPU configuration accordingly.

TABLE II
BEEBS BENCHMARK RESULTS (NO COMPARTMENT MERGING)

Algorithm	General Firmware (Clocks)	REMAP (Clocks)	Overhead (%)	Compartments	MemManage Faults
Aha-Compress	85,398,097	88,093,196	3.15	20	15
Binary Search	479,381	510,513	8.51	19	18
Bubble Sort	546,344,600	563,511,798	3.14	21	16
Cnt	20,068,100	33,898,398	168.91	28	14,009
Compress	21,044,881	664,203,654	3,156.12	48	620,007
Nettle-AES	222,584,261	252,906,239	13.62	37	22,677
Nettle-SHA256	20,927,264	41,830,880	99.88	32	20,010
Cover	13,333,699	13,770,044	3.27	19	13
CRC	9,615,670	9,941,347	3.38	22	20
CRC-32	122,522,925	126,385,139	3.15	20	15
Dijkstra	43,945,447	517,954,386	1,178.63	35	482,072
Edn	311,321,721	344,302,347	10.59	29	24,018
Expint	10,941,725	11,302,687	3.29	21	15
Huffbench	19,126,925	19,943,214	4.26	27	227

Handling a memory fault requires approximately 1037 cycles, corresponding to about 9.46 µs on a 110 MHz Cortex-M33. In absolute terms, this constitutes a very small latency of approximately 0.009% of a 1 ms scheduling tick to resolve a missing memory region. However, the cumulative cost depends on the frequency of such faults, which in turn is influenced by the number of distinct compartments accessed by a task and the frequency of MPU region evictions and reloads.

5) Benchmark Results (BEEBS): To quantify the worst-case overhead in a realistic code, our evaluation was based on the Bristol/Embecosm Embedded Benchmark Suite (BEEBS). BEEBS is a widely used, MCU-oriented benchmark collection that covers diverse computational workloads and memory-access patterns. This diversity allows observation across both typical cases with limited compartment access and stress cases involving frequent cross-compartment transitions. In particular, memory-intensive workloads, such as *Compress* and *Dijkstra* frequently trigger fine-grained compartment changes, amplifying potential overheads and enabling a thorough stress test of REMAP efficiency. Additionally, BEEBS is open-source, lightweight, and easily portable to ARMv8-M/FreeRTOS platforms, which ensures reproducibility and fairness across runs. The algorithms listed in Tables II and III are sourced directly from this suite, and detailed documentation and source code are available in [27], [28]. Accordingly, BEEBS provides an effective combination of realism, coverage, and practicality for evaluating REMAP in embedded and RTOS environments.

Using this suite, the total execution cycles along with the number of MemManage faults triggered for each benchmark were measured with REMAP enabled and compared against baseline execution with MPU protection disabled. Table II presents the results for a fine-grained configuration in which each function and global variable is placed in its own compartment. The performance varies significantly across benchmarks. Many small benchmarks such as *Aha-Compress*, *Binary Search*, and *Bubble Sort* exhibit only modest slowdowns under REMAP with approximately 3%–8% overhead, which correspond to a small number of MemManage faults (e.g., 15 or 18 faults). In such cases, the one-time cost of a few faults has a negligible impact on overall runtime. In contrast, benchmarks involving frequent cross-compartment interactions experienced substantial slowdowns. The worst case was observed in *Compress*, which exhibited a 3156% increase in cycle count representing more than 32× slowdown, under the fine-grained scheme due to 6 20 007 memory access faults. Other memory-intensive benchmarks, such as *Dijkstra* with 1178.6% overhead and 4 82 000 faults, also demonstrate that overly strict compartmentalization can result in excessive fault-handling overhead. These results highlight that while naive fine-grained isolation maximizes security, it can severely degrade performance for tasks with high inter-region accesses.

6) Effect of Compartment Merging: REMAP was further evaluated using an optimized configuration in which certain adjacent memory regions were merged into larger compartments to reduce fault frequency. This optimization preserves isolation semantics by merging only contiguous regions that share identical access permissions, while reducing the total number of MPU regions cycled through at runtime. Table III presents the impact of this conservative merging on the BEEBS benchmarks. The improvement is substantial as all benchmarks exhibited 5% or less overhead. In the previous worst-case example, the *Compress* benchmark slowdown decreased from over 3000% to just 3.19% after clustering numerous small regions. The number of faults for *Compress* dropped from 6 20 000 to only 8 under the merged configuration. Other memory-intensive benchmarks, such as *Dijkstra*, also experienced significant reductions in both fault count and overhead (e.g., the overhead reduced to approximately 3% after merging, which is down from over 1000% without merging). Even lightweight benchmarks showed minor improvements; for example, the overhead of *Binary Search* was reduced from 8.5% to 5.0%. These results confirm that through intelligent region merging, REMAP can achieve near-zero performance impact across a wide range of tasks. In practical settings, developers can tune compartment granularity to balance security isolation and performance. Our results demonstrate that modest relaxation of compartment boundaries significantly reduces worst-case overhead while still enforcing well-defined memory access constraints.

TABLE III
BEEBS BENCHMARK RESULTS (WITH COMPARTMENT MERGING)

Algorithm	General Firmware (Clocks)	REMAP (Clocks)	Overhead (%)	Compartments	MemManage Faults
Aha-Compress	85,398,097	88,015,038	3.06	11	6
Binary Search	479,381	503,412	5.01	12	6
Bubble Sort	546,344,600	563,061,844	3.05	13	8
Cnt	20,068,100	20,693,060	3.11	12	6
Compress	21,044,881	21,716,936	3.19	14	8
Nettle-AES	222,584,261	229,584,629	3.14	20	8
Nettle-SHA256	20,927,264	21,779,534	3.20	17	12
Cover	13,333,699	13,742,074	3.06	10	5
CRC	9,615,670	9,929,747	3.35	14	8
CRC-32	122,522,925	126,377,030	3.14	13	8
Dijkstra	43,945,447	45,342,297	3.17	15	13
Edn	311,321,721	321,106,346	3.14	13	8
Expint	10,941,725	11,285,541	3.14	12	6
Huffbench	19,126,925	19,743,358	3.22	14	13

7) Dynamic Allocation/Deallocation Microbenchmark: To quantify the per-operation overhead introduced by dynamic allocation support of REMAP, a single allocation/deallocation sequence was measured using the provided wrapper APIs. In a baseline environment (i.e., without REMAP), one *malloc()* call required 1019 cycles on average. Under REMAP, the same operation took 1115 cycles, yielding an overhead of 9.42%. The allocator stores 16 bytes of metadata per dynamically allocated block to record its address and size, resulting in an additional 16-byte memory overhead for each allocation. For deallocation, a baseline *free()* required 778 cycles, whereas *REMAP_free()* took 875 cycles (12.46% overhead). These results indicate that the per-call cost of managing ACT entries is modest. However, the cumulative impact depends on the frequency of allocations and deallocations in a given workload.

B. Real-Time Applicability

A key concern for real-time systems is whether dynamic MPU reconfigurations introduced by REMAP could result in unpredictability or unbounded latency. Our evaluation results indicate that the overhead of MPU updates remains consistent and predictable for each task. At most, a task incurs a single MemManage fault upon first access to each authorized memory region. Once a region is loaded into an MPU slot, subsequent accesses proceed at native speed. In addition, REMAP preserves the MPU state across context switches, preventing repeated faults on previously accessed regions. Consequently, the maximum number of faults per task can be estimated through static analysis of its memory access pattern. This leads to a deterministic worst-case overhead per task, which is essential for real-time schedulability analysis. Benchmark results show that no individual memory fault required more than approximately 9.5 µs to resolve, and the number of compartments bounded the fault count per task. The fault frequency decreases significantly when merging is applied, further enhancing predictability.

Another consideration is the potential interference of fault handling with real-time deadlines. Since each MemManage exception momentarily preempts the running task to update permissions, a small and constant execution time is added. This overhead, however, is comparable to that of a fast interrupt service routine and, as shown earlier, can be maintained under 0.01% of typical tick periods per access. By allocating a few microseconds in the worst-case execution time for each memory region access, system designers can ensure that REMAP enforces dynamic checks without violating task deadlines. In practice, tasks often access the same memory regions repeatedly, particularly in periodic control loops, so fault costs are typically incurred only during the first access. As a result, REMAP remains compatible with real-time constraints. Also, future ARM MPU designs offering more than eight regions are expected to further enhance real-time performance. With additional MPU slots, more of the memory map associated with each task can remain preloaded, significantly reducing the frequency of runtime faults in most cases.

C. Comparison With Related Works

Table IV compares REMAP with prior memory protection mechanisms designed for embedded and RTOS environments, which include TrustLite [13], Tytan [14], ACES [10], EPOXY [18], CRT-C [23], MINION [15], CompartOS [16], EC [25], and OPEC [26].

However, it is important to note that directly comparing REMAP with these systems through experiments is often impractical or even misleading. The main reasons are as follows: First, there exist diverse hardware dependencies. Approaches such as TrustLite, Tytan, and CompartOS rely on custom hardware extensions or modified MPU/ISA features, which fundamentally differ from the commercial ARMv8-M microcontrollers used by REMAP. Running identical benchmarks on these heterogeneous platforms would not yield a fair performance comparison. Second, the protection models differ significantly. Systems like EPOXY, ACES, and CRT-C enforce purely static policies fixed at compile time, whereas REMAP dynamically reconfigures the MPU at runtime. Comparing overhead between static and dynamic enforcement schemes can misrepresent the tradeoffs each design targets. Third, many related works lack publicly available prototypes, which makes it infeasible to reproduce experiments under the environment

TABLE IV
COMPARISON WITH RELATED WORK

Approach	Granularity	MPU Management	Dynamic Protection	Hardware Dependency
REMAP	Function/Variable-level	Dynamic	Yes	No
TrustLite [13]	Task-level	Static	No	Yes
TyTan [14]	Task-level	Static	Limited	Yes
CompartOS [16]	Task-level	Dynamic	Yes	Yes
EPOXY [18]	Task-level	Static	No	No
ACES [10]	Task-level	Static	No	No
CRT-C [23]	Variable-level	Static	No	No
EC [25]	Task/File-level	Static	Limited	No
MINION [15]	Task-level	Dynamic	Limited	No
OPEC [26]	Operation-level	Dynamic	Limited	No

of REMAP. Consequently, the evaluation focuses on a qualitative and functional comparison that highlights the protection scope, granularity, MPU management strategy, and hardware requirements of each system, thereby clarifying the unique contributions of REMAP.

To further clarify these differences, it is important to examine the architectural and operational contrasts between REMAP and representative prior systems. Unlike TrustLite and Tytan, which implement hardware-enforced isolation through custom microcontroller architectures, REMAP requires no specialized hardware modifications and operates on standard ARMv8-M microcontrollers. TrustLite and Tytan also rely on predominantly static partitioning of code and data, thus lacks support for runtime privilege adjustments. In contrast, REMAP performs dynamic MPU reconfiguration to adapt to the memory access requirements of each task at runtime.

Approaches such as EPOXY and ACES use static analysis to precompute memory access permissions at compile-time and configure the MPU accordingly. Although these techniques enhance baseline security, their protections are fixed and cannot accommodate dynamic behavior, such as runtime memory allocation, due to the limited MPU slots. Consequently, they often adopt coarse-grained compartmentalization. REMAP also incorporates static analysis to generate initial compartments but extends these with a runtime enforcement layer capable of handling dynamic memory and on-demand region mapping. This hybrid approach offers both safety and flexibility, exceeding the capabilities of purely static systems like EPOXY and ACES.

CRT-C enforces memory safety using compile-time techniques based on Checked C[24] and supports varied granularity of access control, but lacks a runtime enforcement mechanism. As such, CRT-C cannot respond adaptively if task memory usage diverges from statically determined bounds. In contrast, REMAP triggers a fault and grants legitimate access dynamically, enabling flexible enforcement while preserving security guarantees.

Among systems that support dynamic behavior, CompartOS and MINION are particularly notable. CompartOS allows dynamic compartmentalization in an RTOS environment but is built on the CHERI hardware architecture, which introduces capability-based ARM extensions and is therefore not deployable on conventional microcontrollers. Furthermore, CompartOS provides isolation primarily at the task or module level and does not offer fine-grained compartments at the function or variable level.

In contrast, MINION uses the standard ARM MPU dynamically for task isolation. While MINION demonstrated the feasibility of runtime MPU region switching, it retained task-level granularity with each task treated as a monolithic memory region using coarse permissions. As a result, it is unable enforce intratask memory boundaries (e.g., among different components within the same task or between the stack and global variables) and lacks support for protecting dynamically allocated memory. REMAP advances the features of MINION by allowing isolation at a much finer granularity and managing memory access on a per-use basis via its ACT and fault handler.

Likewise, OPEC instruments security-sensitive operations so that the MPU is reconfigured on demand at each operation boundary. By reconfiguring the MPU right before and after each operation, OPEC allows a region to be accessible only when that operation executes. However, its isolation scope is still bounded by operation-level granularity rather than individual data objects, and it does not systematically track heap allocations or persistent intratask regions. Consequently, its dynamic protection remains limited compared with the ACT-driven, per-object management of REMAP.

Finally, EC adopts a hybrid approach that combines LLVM-based static partitioning with runtime monitoring using the DWT unit to detect unauthorized accesses. EC successfully achieves intrakernel isolation and maintains compatibility with existing RTOS code, but its compartment definitions are relatively coarse with each thread or file forming a compartment. Moreover, its runtime component is limited to detection and does not actively manage or reconfigure permissions.

D. Limitation

While the evaluation demonstrates the efficacy of REMAP, certain limitations exist. The experiments were conducted primarily on single-task benchmark scenarios due to the lack of publicly available multitask benchmark suites for embedded RTOS environments. As a result, system behavior under complex multitasking workloads remains to be fully

validated. In particular, compartmentalization may become more fine-grained, potentially leading to benchmark results that deviate from those observed in isolated single-task scenarios. Notably, memory fragmentation can significantly impact the effectiveness of the compartment merging optimization of REMAP. REMAP merges memory regions into a single ACT entry only when they are adjacent and shares identical access permissions. If code or data regions are fragmented in memory or have mismatched permissions, they cannot be merged resulting in more individual compartments. This increases MPU slot usage and may lead to more frequent MemManage faults. To mitigate this, REMAP can leverage linker scripts to guide the placement of code and data into physically contiguous memory regions, thereby improving merge efficiency.

In addition, while REMAP supports dynamic memory allocation, this work does not include a large-scale evaluation under workloads that continuously invoke high-frequency allocation and deallocation operations. Designing such experiments is nontrivial, as memory usage patterns vary significantly across applications, and no standardized RTOS benchmark currently exists to emulate realistic dynamic allocation. Consequently, although the REMAP mechanism for managing heap objects has been functionally validated, a comprehensive quantitative analysis of its impact under highly dynamic scenarios remains as future work.

Finally, REMAP relies on LLVM-based static analysis, which requires access to the source code or the LLVM intermediate representation (IR). This access is critical for constructing the CFG, which is required for building fine-grained ACT entries. However, REMAP cannot be directly applied to closed-source applications, as generating accurate LLVM IR or comparable semantic information from stripped binaries is a challenging task. Supporting such environments would require robust binary analysis and instrumentation techniques to reconstruct control-flow and memory-access patterns, which is beyond the current scope of REMAP.

VI. Conclusion

The proposed REMAP delivers fine-grained, runtime memory protection for RTOS-based embedded systems without requiring custom hardware. Through the combination of LLVM static analysis and on-demand MPU reconfiguration, the REMAP enforces isolation at the granularity of functions, global variables, and heap objects, and supports transparent adaptation to dynamic memory allocation. Our implementation of REMAP on a Cortex-M33 based prototype shows that flash and SRAM overhead is around 5%, context-switch protection costs about 0.65% of CPU time, and a first-access fault completes in roughly 9.5 µs. In addition, conservative region merging reduces the overhead to within 5%. These results confirm that deterministic, fine-grained isolation can be achieved on commodity ARMv8-M devices with negligible impact on real-time responsiveness. The remaining challenges include validating its practicality in complex multitasking applications, exploring smarter region-replacement and layout heuristics to more effectively reduce first-use faults, and leveraging upcoming MPUs with larger region counts to minimize worst-case overheads.

References

[1] B. Blumenscheid. “10 real life examples of embedded systems.” 2021. [Online]. Available: https://www.digi.com/blog/post/examples-of-embedded-systems

[2] A. V. Manoj. “Embedded security: Challenges and strategies.” 2024. [Online]. Available: https://experionglobal.com/embedded-security

[3] D. Papp, Z. Ma, and L. Buttyan, “Embedded systems security: Threats, vulnerabilities, and attack taxonomy,” in *Proc. 13th Annu. Conf. Privacy, Secur. Trust (PST)*, 2015, pp. 145–152.

[4] J. Maslow. “Top emerging threats in embedded systems security and how to mitigate them.” 2025. [Online]. Available: https://usawire.com/top-emerging-threats-in-embedded-systems-security-and-how-to-mitigate-them/

[5] V. Cheptsov and A. Khoroshilov, “Robust resource partitioning approach for ARINC 653 RTOS,” in *Proc. Ivannikov Ispras Open Conf. (ISPRAS)*, 2023, pp. 33–39.

[6] N. Dejon, C. Gaber, and G. Grimaud, “From MMU to MPU: adaptation of the Pip kernel to constrained devices,” 2023, *arXiv:2301.04546*.

[7] W. Zhou, L. Guan, P. Liu, and Y. Zhang “Good motive but bad design: Why ARM MPU has become an outcast in embedded systems,” 2019, *arXiv:1908.03638*.

[8] V. E. Moghadam, M. Meloni, and P. Prinetto, “Control-flow integrity for real-time operating systems: Open issues and challenges,” in *Proc. IEEE East-West Design Test Symp. (EWDTS)*, 2021, pp. 1–6.

[9] J. Sandin. “Exploiting memory corruption vulnerabilities on the FreeRTOS OS.” 2019. [Online]. Available: https://shmoo.gitbook.io/2016-shmoocon-proceedings/bring_it_on/01_exploiting_memory_corruption

[10] A. A. Clements, N. S. Almakhdhub, S. Bagchi, and M. Pay, “ACES: Automatic compartments for embedded systems,” in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 1–19.

[11] A. Mera, Y. H. Chen, R. Sun, E. Kirda, and L. Lu “D-box: DMA-enabled compartmentalization for embedded applications,” 2022, *arXiv:2201.05199*.

[12] H. Lefeuvre, N. Dautenhahn, D. Chisnall, and P. Olivier, “SoK: Software compartmentalization,” 2024, *arXiv:2410.08434*.

[13] P. Koeberl, S. Schulz, A. R. Sadeghi, and V. Varadharajan, “TrustLite: A security architecture for tiny embedded devices,” in *Proc. 9th Eur. Conf. Comput. Syst.*, 2014, pp. 1–14.

[14] F. Brasser, B. El Mahjoub, A. R. Sadeghi, C. Wachsmann, and P. Koeberl, “TyTAN: Tiny trust anchor for tiny devices,” in *Proc. 52nd Annu. Design Autom. Conf.*, 2015, pp. 1–6.

[15] C. H. Kim et al. “Securing real-time microcontroller systems through customized memory view switching,” in *Proc. NDSS*, 2018, pp. 1–15.

[16] H. Almatary et al. “CompartOS: CHERI compartmentalization for embedded systems,” 2022, *arXiv:2206.02852*.

[17] R. N. M. Watso et al. “CHERI: A hybrid capability-system architecture for scalable software compartmentalization,” in *Proc. IEEE Symp. Security Privacy*, 2015, pp. 20–37.

[18] A. A. Clements et al., “Protecting bare-metal embedded systems with privilege overlays,” in *Proc. IEEE Symp. Security Privacy (SP)*, 2017, pp. 289–303.

[19] “Arm® architecture reference manual Armv8, for Armv8-M architecture profile.” Arm. 2017. [Online]. Available: https://developer.arm.com/

[20] J. Yiu, *The Definitive Guide to the ARM Cortex-M3*. Amsterdam, The Netherlands: Newnes, 2009.

[21] (Real Time Eng. Ltd., Kolkata, India). *Memory Protection Unit (MPU) Support*. 2025. [Online]. Available: https://www.freertos.org/Security/04-FreeRTOS-MPU-memory-protection-unit

[22] W. Zhou, Z. Jiang, and L. Guan, “Understanding MPU usage in microcontroller-based systems in the wild,” in *Proc. Workshop Binary Anal. Res.*, San Diego, CA, USA, 2023, pp. 1–7.

[23] A. Khan, D. Xu, and D. J. Tian, “Low-cost privilege separation with compile time compartmentalization for embedded systems,” in *Proc. IEEE Symp. Security Privacy (SP)*, 2023, pp. 3008–3025.

[24] A. S. Elliott, A. Ruef, M. Hicks, and D. Tarditi, “Checked C: Making C safe by extension,” in *Proc. IEEE Cybersecur. Develop. (SecDev)*, 2018, pp. 53–60.

[25] A. Khan, D. Xu, and D. J. Tian, “EC: Embedded systems compartmentalization via intra-kernel isolation,” in *Proc. IEEE Symp. Security Privacy (SP)*, 2023, pp. 2990–3007.

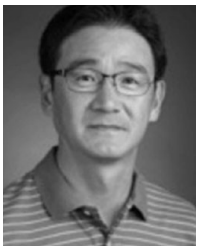
[26] X. Zhou, J. Li, W. Zhang, Y. Zhou, W. Shen, and K. Ren, "OPEC: Operation-based security isolation for bare-metal embedded systems," in *Proc. 17th Eur. Conf. Comput. Syst.*, 2022, pp. 317–333.
[27] J. Pallister, S. Hollis, and J. Bennett, "BEEBS: Open benchmarks for energy measurements on embedded platforms," 2013, *arXiv:1308.5174*.
[28] J. Bennett. "mageec/beebs." 2013. [Online]. Available: https://github.com/mageec/beebs

Heeseung Son received the B.S. and M.S. degrees from the Department of Computer Engineering, Kyung Hee University, Seoul, South Korea, in 2020 and 2022, respectively, where he is currently pursuing the Ph.D. degree with the Department of Computer Engineering.

His research interests include embedded systems and system security.

BeomSeok Kim received the B.S., M.S., and Ph.D. degrees from the Department of Computer Engineering from Kyung Hee University, Seoul, South Korea, in 2010, 2012, and 2016, respectively.

He was an Assistant Professor with the Department of Computer Software Engineering, Changshin University, Changwon, South Korea, from 2016 to 2018. He was a Research Professor with the Department of Computer Engineering, KyungHee University, from 2018 to 2025. Currently, he is an Assistant Professor with the Department of Artificial Intelligence and Software Engineering, Major in Information Security, Chosun University, Gwangju, South Korea. His research interests include wireless sensor and body area networks, embedded systems, and network security.



Ben Lee received the B.E. degree in electrical engineering from the Department of Electrical Engineering, State University of New York, Stony Brook, NY, USA, in 1984, and the Ph.D. degree in computer engineering from the Department Electrical and Computer Engineering, Pennsylvania State University, University Park, PA, USA, in 1991.

His research interests include multimedia streaming, wireless networks, embedded systems, computer architecture, multithreading and thread-level speculation, and parallel and distributed systems.

Dr. Lee was a recipient of the Loyd Carter Award for Outstanding and Inspirational Teaching in 1994, the Alumni Professor Award for Outstanding Contribution to the College and the University from the OSU College of Engineering in 2005, and the HKN Innovation Teaching Award from Eta Kappa Nu, School of Electrical Engineering and Computer Science in 2008. He has been on the program committees and organizing committee for numerous international conferences, including the 2005?2012 IEEE Workshop on Pervasive Wireless Networking, and the IEEE International Conference on Pervasive Computing and Communications (PerCom). He was the Workshop Chair for PerCom 2009. He was a Guest Editor for the Special Issue on Wireless Networks and Pervasive Computing of the *Journal of Pervasive Computing and Communications*. He was also an Invited Speaker at the 2007 International Conference on Embedded Software and System and a Keynote Speaker at the 2014 ACM International Conference on Ubiquitous Information Management and Communication. He was the TPC Chair at the 15th Annual IEEE Consumer Communications and Networking Conference (CCNC 2018). He is also an Adjunct Faculty Member from the Korea Advanced Institute of Science and Technology.

Jinsung Cho (Member, IEEE) received the B.S., M.S., and Ph.D. degrees from Seoul National University, Seoul, South Korea, in 1992, 1994, and 2000, respectively, all in computer engineering.

He was a Visiting Researcher with the IBM T. J. Watson Research Center in 1998, and a member of the research staff with Samsung Electronics from 1999 to 2003. He is currently a Professor with the Department of Computer Engineering, Kyung Hee University, Seoul. His research interests include mobile system security, embedded security, IoT security, and sensor and body networks.